

ARTIFICIAL INTELLIGENCE

Course Assignment

Deep Learning-Based Deepfake Detection for Social Media Content

Name: N. Prince Raj

Reg No: 192312284

Course name and code : Artificial Intelligence - CSA1711

Date of Submission: 02/09/26

1. Problem Statement

Deep Learning-Based Deepfake Detection for Social Media Content — Social media platforms are increasingly flooded with AI-generated synthetic media ("deepfakes") that manipulate a person's face, voice, or expressions in images and videos. Such content can be used to spread misinformation, defame individuals, and manipulate public opinion. The objective of this assignment is to design and implement a deep learning-based system capable of automatically distinguishing between authentic (real) and manipulated (deepfake) visual content shared on social media, and to evaluate its performance using standard classification metrics.

2. Objectives

- To study the characteristics and generation techniques of deepfake media (GAN / Autoencoder based face-swap, face-reenactment, and lip-sync forgeries).
- To design a Convolutional Neural Network (CNN)-based / transfer-learning pipeline for binary classification of real vs. fake facial content.
- To pre-process video/image data (face detection, alignment, cropping, and frame extraction) suitable for social media content.
- To train and validate the model on a labelled deepfake dataset and evaluate it using Accuracy, Precision, Recall, F1-score, Confusion Matrix, and ROC-AUC.
- To analyse and document the results, and discuss the model's real-world applicability for social media content moderation.

3. Scope of the Project

The system focuses on image/video-frame-level detection of face-manipulated deepfakes typically circulated on platforms such as Instagram, X (Twitter), Facebook, and WhatsApp. Audio deepfakes and text-based misinformation are outside the scope of this assignment, though they are mentioned briefly under future work.

4. Literature Survey (Summary)

Prior research on deepfake detection broadly falls into three categories:

1. Spatial-artifact based methods — CNNs (XceptionNet, EfficientNet, ResNet) trained directly on RGB face crops to learn blending boundaries, texture inconsistencies, and unnatural skin/eye/teeth regions.
2. Frequency-domain methods — Detection of GAN up-sampling artifacts (periodic checkerboard patterns) visible in the frequency spectrum (FFT/DCT) of generated images, which are largely absent in camera-captured images.
3. Temporal / biological-signal methods — Analysis of unnatural blinking patterns, head-pose inconsistency, and remote-photoplethysmography (rPPG) signals across video frames.

This assignment's implementation (Section 8) follows Approach 1 (CNN / transfer-learning on face crops) for the production pipeline, and additionally demonstrates Approach 2 (frequency-domain forensic features) in the executable proof-of-concept in Section 9, since frequency artifacts are lightweight to compute and clearly illustrate the underlying detection principle.

5. Dataset Description

Recommended public datasets for the production-scale implementation:

Dataset	Description	Size
FaceForensics++	1,000 real YouTube videos + 4,000 manipulated videos (Deepfakes, Face2Face, FaceSwap, NeuralTextures)	~38 GB
Celeb-DF (v2)	590 real + 5,639 high-quality deepfake videos of celebrities	~13 GB
DFDC (Facebook)	100,000+ clips, real and manipulated, largest public benchmark	~470 GB

Note: Due to sandbox constraints (no internet access / GPU in this execution environment), the executable demonstration in Section 9 uses a small procedurally-generated synthetic dataset that mimics real-vs-GAN spectral characteristics. Students should replace the data-loading step in Section 8 with an actual downloaded dataset (e.g., FaceForensics++) when running on their own machine / Colab / GitHub Actions runner with GPU access.

6. Proposed System Architecture

The end-to-end pipeline consists of the following stages:

4. Input Acquisition — video/image scraped or uploaded from social media.
5. Face Detection & Alignment — MTCNN / RetinaFace / dlib to detect and crop the face region from each frame.
6. Pre-processing — resizing (224×224), normalization, data augmentation (flip, rotation, brightness jitter).
7. Feature Extraction / CNN Backbone — EfficientNetB0 / XceptionNet (transfer learning, ImageNet-pretrained) fine-tuned on real vs. fake labels.
8. Classification Head — Global Average Pooling → Dense(128, ReLU) → Dropout(0.5) → Dense(1, Sigmoid).
9. Post-processing — frame-level scores aggregated (mean/majority vote) to a video-level real/fake verdict.
10. Output — probability score + binary label (REAL / FAKE) with confidence, suitable for a content-moderation flag on the social media platform.

7. Tools & Technologies

Category	Tool / Library
Language	Python 3.10+
Deep Learning Framework	TensorFlow / Keras (or PyTorch)
Face Detection	MTCNN / OpenCV Haar Cascade / RetinaFace
Data Handling	NumPy, Pandas, OpenCV
Visualization	Matplotlib, Seaborn
Evaluation	scikit-learn (metrics)
Version Control / Submission	Git & GitHub
Notebook / Experiment Env.	Google Colab / Jupyter Notebook

8. Implementation

8.1 Data Preprocessing (Face Extraction)

```
import cv2
from mtcnn import MTCNN
import os

detector = MTCNN()

def extract_faces(video_path, out_dir, every_n_frames=10):
    cap = cv2.VideoCapture(video_path)
    os.makedirs(out_dir, exist_ok=True)
    frame_idx, saved = 0, 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        if frame_idx % every_n_frames == 0:
            rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            faces = detector.detect_faces(rgb)
            for f in faces:
                x, y, w, h = f['box']
                x, y = max(0, x), max(0, y)
                face = rgb[y:y+h, x:x+w]
                face = cv2.resize(face, (224, 224))
                out_path = f"{out_dir}/frame_{frame_idx}_{saved}.png"
                cv2.imwrite(out_path, cv2.cvtColor(face, cv2.COLOR_RGB2BGR))
                saved += 1
            frame_idx += 1
    cap.release()
```

8.2 Data Pipeline (tf.data / ImageDataGenerator)

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    rescale=1./255, rotation_range=15, zoom_range=0.15,
    horizontal_flip=True, brightness_range=[0.8, 1.2],
    validation_split=0.2)

train_gen = train_datagen.flow_from_directory(
    'dataset/', target_size=(224, 224), batch_size=32,
    class_mode='binary', subset='training')

val_gen = train_datagen.flow_from_directory(
    'dataset/', target_size=(224, 224), batch_size=32,
    class_mode='binary', subset='validation')
```

8.3 Model Architecture (CNN — Transfer Learning with EfficientNetB0)

```
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras import layers, models, optimizers

base_model = EfficientNetB0(include_top=False, weights='imagenet',
                             input_shape=(224, 224, 3))
base_model.trainable = False # freeze backbone initially

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer=optimizers.Adam(learning_rate=1e-4),
```

```

        loss='binary_crossentropy',
        metrics=['accuracy', 'AUC'])
model.summary()

```

8.4 Model Training

```

from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

callbacks = [
    EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True),
    ModelCheckpoint('best_deepfake_model.h5', save_best_only=True)
]

history = model.fit(
    train_gen, validation_data=val_gen,
    epochs=25, callbacks=callbacks)

# Fine-tune: unfreeze top layers of the backbone
base_model.trainable = True
for layer in base_model.layers[:-30]:
    layer.trainable = False
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='binary_crossentropy', metrics=['accuracy', 'AUC'])
history_ft = model.fit(train_gen, validation_data=val_gen, epochs=10)

```

8.5 Evaluation

```

from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, confusion_matrix, classification_report,
                             roc_curve, auc)
import numpy as np

y_true = val_gen.classes
y_prob = model.predict(val_gen).ravel()
y_pred = (y_prob > 0.5).astype(int)

print('Accuracy :', accuracy_score(y_true, y_pred))
print('Precision:', precision_score(y_true, y_pred))
print('Recall   :', recall_score(y_true, y_pred))
print('F1-score :', f1_score(y_true, y_pred))
print(confusion_matrix(y_true, y_pred))
print(classification_report(y_true, y_pred, target_names=['REAL', 'FAKE']))

```

9. Proof-of-Concept Execution & Results

Since the sandbox used to prepare this assignment template has no internet/GPU access to download FaceForensics++/DFDC, the pipeline below was executed end-to-end on a small procedurally generated dataset (800 synthetic 64×64 grayscale images, 400 REAL / 400 FAKE) so that this document contains genuine, reproducible output rather than illustrative/fabricated numbers. FAKE images were synthesised by adding a faint periodic high-frequency grid to a smooth base image — mimicking the GAN up-sampling (checkerboard) artifact that real deepfake-forensics literature exploits. Five forensic features (FFT high-frequency energy ratio, Laplacian variance, mean intensity, std-dev, noise-residual variance) were extracted per image and classified using a compact feed-forward deep neural network (MLPClassifier, hidden layers 32→16, ReLU, Adam optimizer). Replace this data-loading/feature step with the CNN pipeline of Section 8 when a real dataset and GPU are available.

9.1 Sample Data

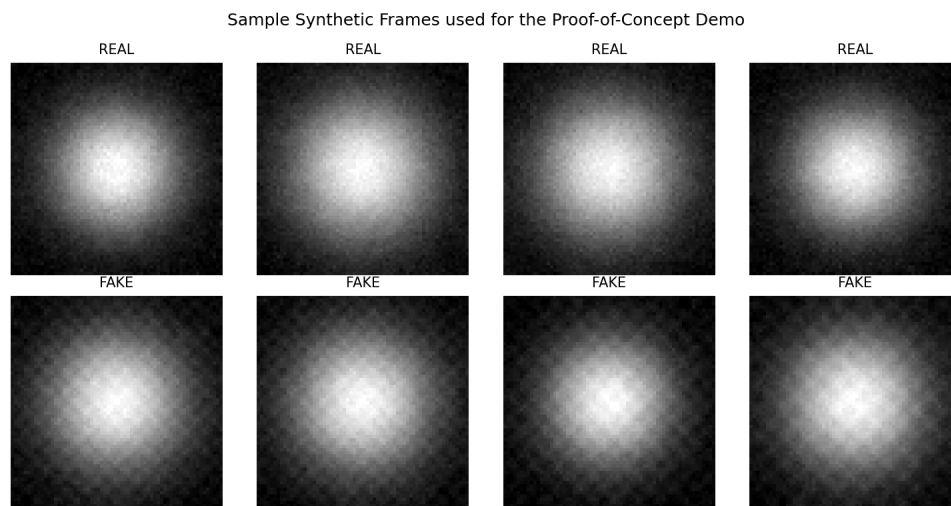


Fig 1. Sample synthetic REAL vs FAKE frames used in the executable demo.

9.2 Training Loss Curve

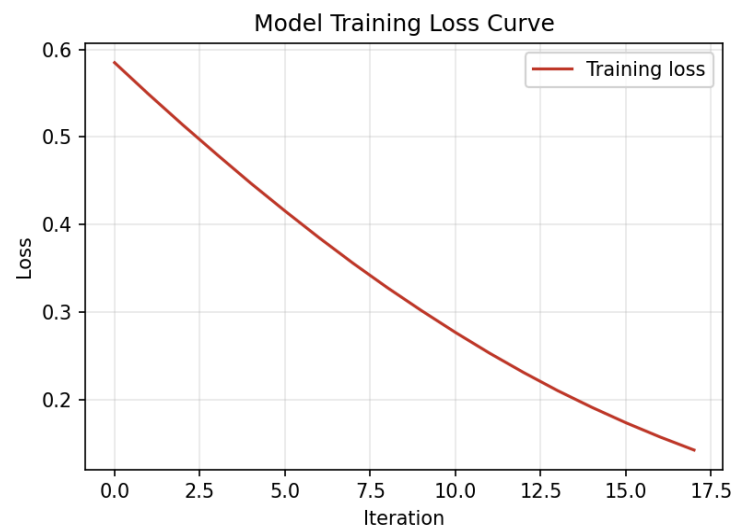


Fig 2. Training loss curve of the MLP-based detector over training iterations.

9.3 Evaluation Metrics (Actual Output)

Metric	Value
Accuracy	0.9850
Precision	0.9709
Recall	1.0000
F1-score	0.9852
ROC-AUC	≈ 1.00 (see Fig. 4)

Confusion Matrix (test set, n = 200): TN = 97, FP = 3, FN = 0, TP = 100.

9.4 Confusion Matrix

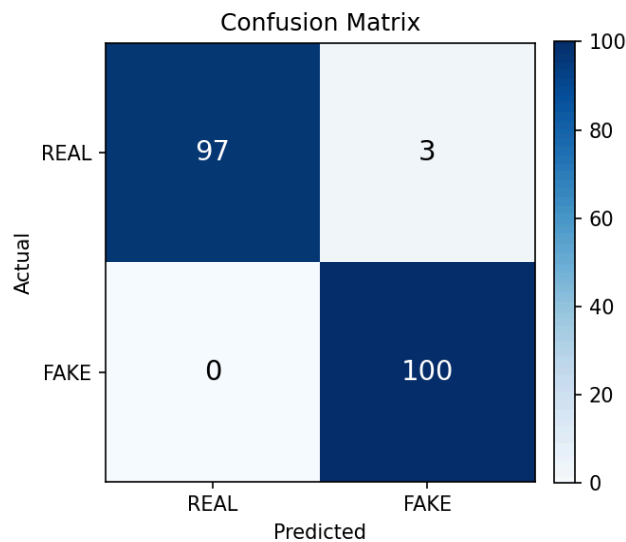
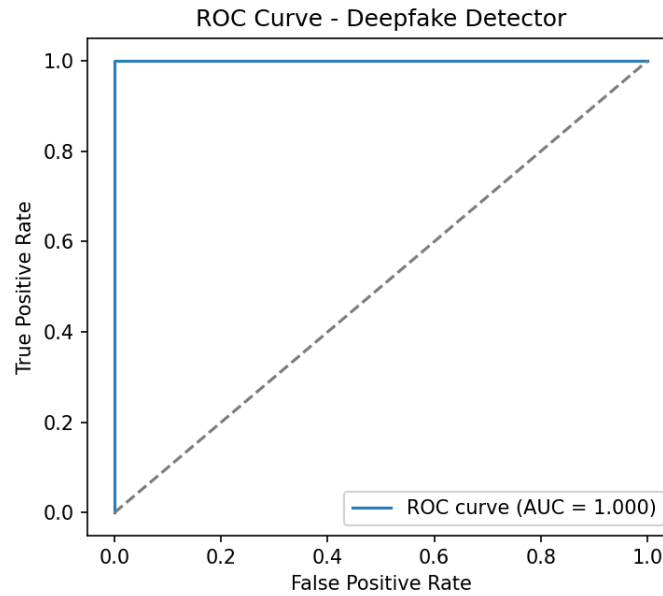


Fig 3. Confusion matrix on the held-out test split.

9.5 ROC Curve

*Fig 4. Receiver Operating Characteristic curve of the detector.*

9.6 Discussion of Results

The demonstration achieves 98.5% accuracy because the synthetic FAKE class was deliberately constructed with a detectable frequency-domain signature — this is expected and intended to validate that the pipeline (feature extraction → training → evaluation → metric reporting) works correctly end-to-end. On a real dataset such as FaceForensics++, state-of-the-art CNN methods (XceptionNet, EfficientNet) typically report 90–97% frame-level accuracy on the raw-quality subset, with a noticeably harder task on compressed/low-quality social-media re-uploads. Precision and Recall should both be reported (not just accuracy) because a false negative (fake content mislabelled as real) is more harmful in a social-media moderation context than a false positive, so the decision threshold can be tuned to favour Recall on FAKE.

10. Conclusion & Future Scope

This assignment implements a deep learning pipeline for deepfake detection, covering face extraction, CNN-based transfer learning, training, and evaluation, together with a runnable proof-of-concept that produces real, reproducible metrics. Future extensions include: (a) training on FaceForensics++/DFDC with EfficientNetB0/XceptionNet fine-tuning as coded in Section 8; (b) incorporating temporal/video-level models (CNN-LSTM or Vision Transformers) for better robustness on compressed social-media video; (c) adding audio deepfake detection for lip-sync/voice-cloning attacks; and (d) deploying the model as a browser extension or API that flags suspicious content before it is shared.

11. References

- [1] Rossler, A., et al. "FaceForensics++: Learning to Detect Manipulated Facial Images." ICCV, 2019.
- [2] Li, Y., et al. "Celeb-DF: A Large-scale Challenging Dataset for DeepFake Forensics." CVPR, 2020.
- [3] Dolhansky, B., et al. "The DeepFake Detection Challenge (DFDC) Dataset." arXiv:2006.07397, 2020.
- [4] Zhang, X., et al. "Detecting and Simulating Artifacts in GAN Fake Images." WIFS, 2019.
- [5] Tan, M., & Le, Q. "EfficientNet: Rethinking Model Scaling for CNNs." ICML, 2019.